(b) $E \vdash p \diamond w$ *("p is independent of w after E") and* $v' \sim_{[E.p]} w$.

Proof. We examine each case separately.

(1) If $v = \langle\rangle$, let $v' = w$ and $w' = \langle\rangle$. Then $E.v.v' \simeq E.v' \simeq E.w \simeq E.w.w'$, hence $v \sim_{[E]} w$.
(2) If $v = p.v'$ then:
   (a) If $p \in I_{[E]}(w)$ and $v' \sim_{[E.p]} (w \setminus p)$, then by $p \in I_{[E]}(w)$ and Definition 4.1(1) we get that there exists w' such that $E.w \simeq E.p.w'$. In the proof of that Lemma (forward direction), it is easy to see that such a w' is exactly $w \setminus p$, therefore $E.w \simeq E.p.(w \setminus p)$. By $v' \sim_{[E.p]} (w \setminus p)$ we get that there exist sequences v'' and w'' such that $E.v.v'' \simeq E.p.v'.v'' \simeq E.p.(w \setminus p).w'' \simeq E.w.w''$, hence $v \sim_{[E]} w$.
   (b) If $E \vdash p \diamond w$ and $v' \sim_{[E.p]} w$, then from $E \vdash p \diamond w$ we have that $E.p.w \simeq E.w.p$. From $v' \sim_{[E.p]} w$ we also have that there exist v'' and w' such that $E.v.v'' \simeq E.p.v'.v'' \simeq E.p.w.w' \simeq E.w.p.w'$, hence $v \sim_{[E]} w$.

□

The following lemma states some useful properties.

Lemma 4.7. *Let E be an execution sequence, and let v, w, and w′ be sequences. Then*

(1) $E.w' \simeq E.w$ *implies that (i)* $v \sqsubseteq_{[E]} w$ *iff* $v \sqsubseteq_{[E]} w'$*, and (ii)* $w \sqsubseteq_{[E]} v$ *iff* $w' \sqsubseteq_{[E]} v$*, and (iii)* $v \sim_{[E]} w$ *iff* $v \sim_{[E]} w'$*;*
(2) $v \sqsubseteq_{[E]} w$ *and* $w \sim_{[E]} w'$ *imply* $v \sim_{[E]} w'$*;*
(3) $v \sqsubseteq_{[E]} w'$ *and* $w \sqsubseteq_{[E]} w'$ *imply* $v \sim_{[E]} w$*;*
(4) $p \in WI_{[E]}(w)$ *and* $w' \sqsubseteq_{[E]} w$ *imply* $p \in WI_{[E]}(w')$*;*
(5) $p \in WI_{[E]}(w)$ *and* $E \vdash p \diamond q$ *and* $E \vdash q \diamond w$ *imply* $p \in WI_{[E]}(q.w)$.

All the above properties follow easily from the definitions, so we omit their proofs.

5 SOURCE-DPOR

Having established the concept of source sets, Algorithm 1 presents the *source-DPOR* algorithm. As mentioned in the introduction, this algorithm is derived from the classical persistent-set-based DPOR algorithm of Flanagan and Godefroid [2005b] by replacing persistent sets by source sets. The modification only consists in a small change to a single test in the algorithm. Recall that, as mentioned at the end of Section 3.1, in this section we assume that processes cannot disable each other; in Section 8 we show how the algorithm is extended to the case where process can disable each other.

5.1 Algorithm

Source-DPOR uses the recursive procedure *Explore*(E, *Sleep*) to perform a depth-first search, where E can be interpreted as the stack of the search, i.e., the past execution sequence explored so far and *Sleep* is a sleep set, i.e., a set of processes that (provably) need not be explored from $s_{[E]}$. For each prefix $E' \leq E$, the algorithm maintains a set *backtrack*(E') of processes that will eventually be explored from E', allowing additions to any backtrack set of a state in the stack to be made by recursive calls to *Explore*.

Explore(E, *Sleep*) initializes *backtrack*(E) to consist of an arbitrary enabled process which is not in *Sleep* (line 3). Thereafter, for each process p in *backtrack*(E) which is not in *Sleep*, the algorithm performs two phases: race detection (lines 5–9) and state exploration (lines 10–12).

In the race detection phase, the algorithm first finds the events $e \in dom(E)$ that are in a reversible race with the next step of p (line 5). For each such event $e \in dom(E)$, the algorithm must explore